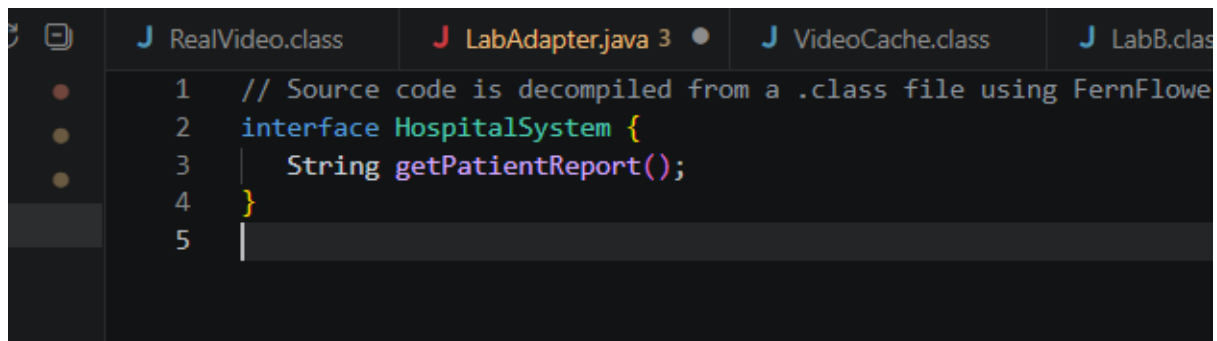


NAME :Trishul Swamy HM
SRN :PES1UG23CS658
OOAD _LAB

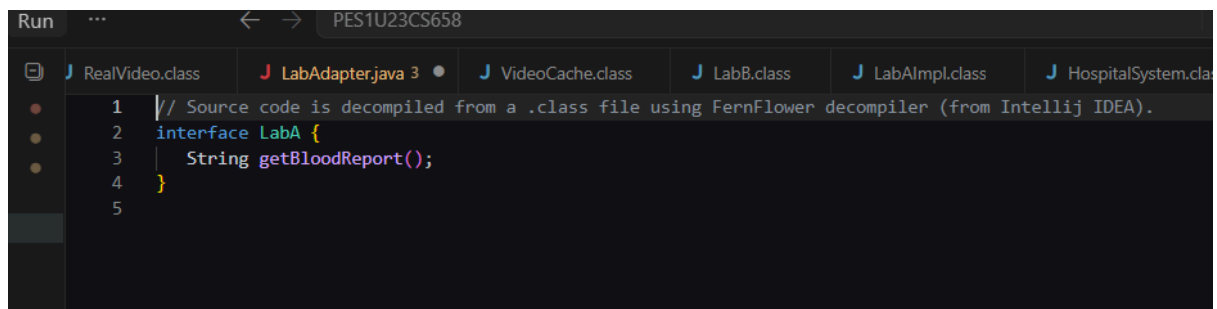
Problem 1

Classes



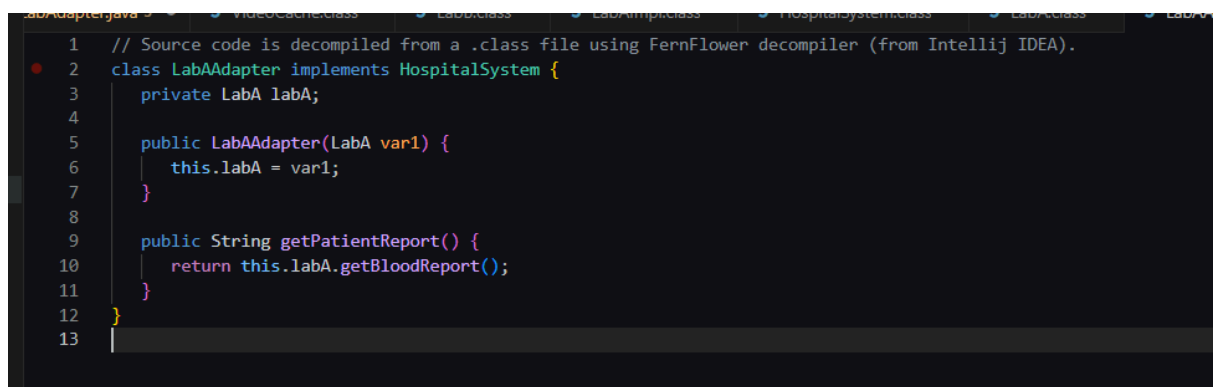
This screenshot shows an IDE window with the file 'LabAdapter.java 3' selected. The code defines an interface named 'HospitalSystem' with a single method 'getPatientReport()' that returns a 'String'.

```
1 // Source code is decompiled from a .class file using FernFlowe
2 interface HospitalSystem {
3     String getPatientReport();
4 }
5
```



This screenshot shows an IDE window with the file 'LabAdapter.java 3' selected. The code defines an interface named 'LabA' with a single method 'getBloodReport()' that returns a 'String'.

```
1 // Source code is decompiled from a .class file using FernFlowe decompiler (from IntelliJ IDEA).
2 interface LabA {
3     String getBloodReport();
4 }
5
```



This screenshot shows an IDE window with the file 'LabAdapter.java 3' selected. The code defines a class 'LabAAdapter' that implements the 'HospitalSystem' interface. It has a private field 'labA' of type 'LabA', a constructor that takes a 'LabA' object and assigns it to 'labA', and a 'getPatientReport()' method that returns the result of 'this.labA.getBloodReport()'.

```
1 // Source code is decompiled from a .class file using FernFlowe decompiler (from IntelliJ IDEA).
2 class LabAAdapter implements HospitalSystem {
3     private LabA labA;
4
5     public LabAAdapter(LabA var1) {
6         this.labA = var1;
7     }
8
9     public String getPatientReport() {
10         return this.labA.getBloodReport();
11     }
12 }
13
```

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 class LabCImpl implements LabC {
3     LabCImpl() {
4     }
5
6     public String retrieveDiagnostics() {
7         return "Lab C - Diagnostics: BP: 120/80, Heart Rate: 72bpm, Temperature: 98.6F";
8     }
9 }
10
```

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 class LabCAdapter implements HospitalSystem {
3     private LabC labC;
4
5     public LabCAdapter(LabC var1) {
6         this.labC = var1;
7     }
8
9     public String getPatientReport() {
10        return this.labC.retrieveDiagnostics();
11    }
12 }
13
```

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 interface LabC {
3     String retrieveDiagnostics();
4 }
5
```

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 class LabBImpl implements LabB {
3     LabBImpl() {
4     }
5
6     public String fetchLabResults() {
7         return "Lab B - Results: Glucose: 95mg/dL, Cholesterol: 180mg/dL, Triglycerides: 100mg/dL";
8     }
9 }
10
```

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 interface LabB {
3     String fetchLabResults();
4 }
5
```

```
J LabAdapter.java 3 J LabAdapter.class X J VideoCache.class J LabB.class J LabAImpl.class J HospitalSystem.class J LabAImpl.class
1 // Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).
2 public class LabAdapter {
3     public LabAdapter() {
4     }
5
6     public static void main(String[] var0) {
7         System.out.println("===== HOSPITAL MANAGEMENT SYSTEM =====");
8         System.out.println("Pattern: ADAPTER PATTERN");
9         System.out.println("Location: hospital_management_system");
10        System.out.println("SRN: PES1U23CS658");
11        System.out.println();
12        LabAImpl var1 = new LabAImpl();
13        LabBImpl var2 = new LabBImpl();
14        LabCImpl var3 = new LabCImpl();
15        LabAAdapter var4 = new LabAAdapter(var1);
16        LabBAdapter var5 = new LabBAdapter(var2);
17        LabCAdapter var6 = new LabCAdapter(var3);
18        System.out.println("Patient Report from Lab A:");
19        System.out.println(var4.getPatientReport());
20        System.out.println();
21        System.out.println("Patient Report from Lab B:");
22        System.out.println(var5.getPatientReport());
23        System.out.println();
24        System.out.println("Patient Report from Lab C:");
25        System.out.println(var6.getPatientReport());
26        System.out.println();
27        System.out.println("===== ADVANTAGES =====");
28        System.out.println("1. Unified Interface: Hospital system can use single getPatientReport() method");
29        System.out.println("2. No Modification: Existing lab classes remain unchanged");
30        System.out.println("3. Flexible: Easy to add new labs by creating new adapters");
31        System.out.println("4. Decoupling: Hospital system decoupled from lab implementations");
32    }
33 }
34
```

Java code :

```
J RealVideo.class J LabAdapter.java 3
hospital_management_system > J LabAdapter.java > java > LabAdapter
1 // Adapter Pattern - Hospital Management System
2 // Design Pattern to convert different lab method calls into unified format
3
4 // Step 1: Create interfaces for different labs
5 interface LabA {
6     String getBloodReport();
7 }
8
9 interface LabB {
10    String fetchLabResults();
11 }
12
13 interface LabC {
14    String retrieveDiagnostics();
15 }
16
17 // Step 2: Create concrete implementations of different labs
18 class LabAImpl implements LabA {
19     @Override
20     public String getBloodReport() {
21         return "Lab A - Blood Report: RBC: 4.5K/uL, WBC: 7K/uL, Hemoglobin: 13.5g/dL";
22     }
23 }
24
25 class LabBImpl implements LabB {
26     @Override
27     public String fetchLabResults() {
28         return "Lab B - Results: Glucose: 95mg/dL, Cholesterol: 180mg/dL, Triglycerides: 100mg/dL";
29     }
30 }
31
32 class LabCImpl implements LabC {
33     @Override
34     public String retrieveDiagnostics() {
35         return "Lab C - Diagnostics: BP: 120/80, Heart Rate: 72bpm, Temperature: 98.6F";
36     }
37 }
38
39 // Step 3: Target interface expected by hospital system
40 interface HospitalSystem {
41    String getPatientReport();
42 }
43
44 // Step 4: Adapter classes - Convert different lab interfaces to hospital system interface
45 class LabAAdapter implements HospitalSystem {
46     private LabA labA;
47
48     public LabAAdapter(LabA labA) {
49         this.labA = labA;
50     }
51
52     @Override
53     public String getPatientReport() {
54         return labA.getBloodReport();
55     }
56 }
```

```

hospital_management_system > J LabAdapter.java > Java > LabAdapter
45 class LabAAdapter implements HospitalSystem {
46
47     @Override
48     public String getPatientReport() {
49         return labA.getBloodReport();
50     }
51 }
52
53 class LabBAdapter implements HospitalSystem {
54     private LabB labB;
55
56     public LabBAdapter(LabB labB) {
57         this.labB = labB;
58     }
59
60     @Override
61     public String getPatientReport() {
62         return labB.fetchLabResults();
63     }
64 }
65
66 class LabCAdapter implements HospitalSystem {
67     private LabC labC;
68
69     public LabCAdapter(LabC labC) {
70         this.labC = labC;
71     }
72
73     @Override
74     public String getPatientReport() {
75         return labC.retrieveDiagnostics();
76     }
77 }
78
79 // Step 5: Main class to demonstrate the pattern
80 public class LabAdapter {
81     Run main | Debug main | Run | Debug
82     public static void main(String[] args) {
83         System.out.println(x: "===== HOSPITAL MANAGEMENT SYSTEM =====");
84         System.out.println(x: "Pattern: ADAPTER PATTERN");
85         System.out.println(x: "Location: hospital_management_system");
86         System.out.println(x: "SRN: PESIU23C5658");
87         System.out.println();
88
89         // Create lab instances
90         LabA labA = new LabImpl();
91         LabB labB = new LabImpl();
92         LabC labC = new LabCImpl();
93
94         // Adapt them to hospital system interface
95         HospitalSystem patientReportA = new LabAAdapter(labA);
96         HospitalSystem patientReportB = new LabBAdapter(labB);
97         HospitalSystem patientReportC = new LabCAdapter(labC);
98
99         // Use unified interface
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122

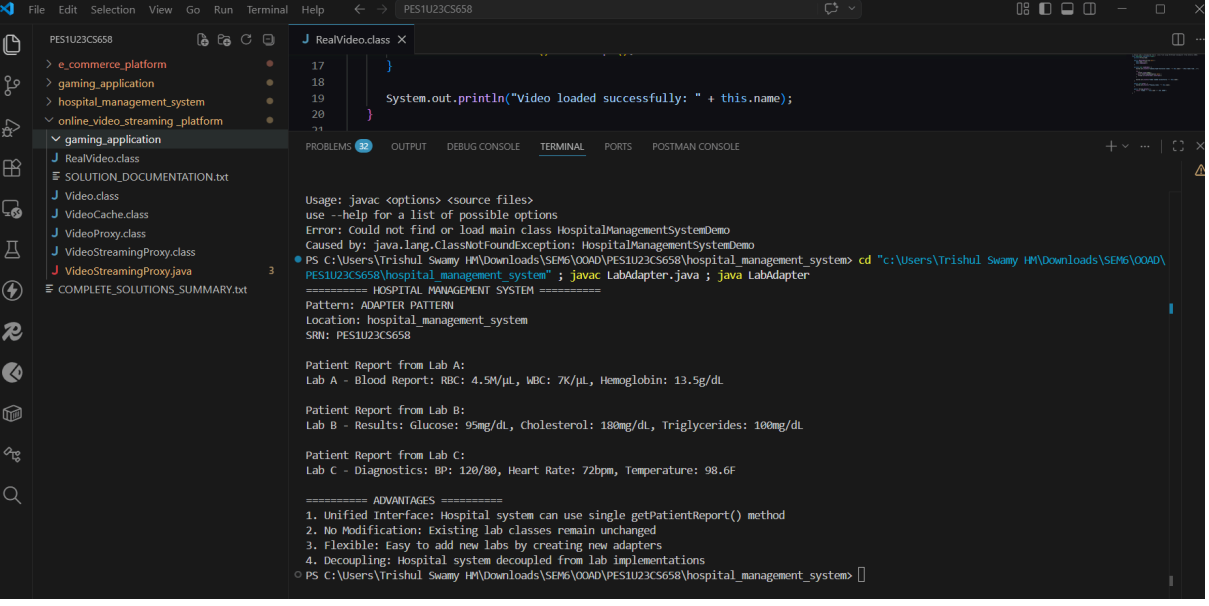
```

```

101     HospitalSystem patientReportC = new LabCAdapter(labC);
102
103     // Use unified interface
104     System.out.println(x: "Patient Report from Lab A:");
105     System.out.println(patientReportA.getPatientReport());
106     System.out.println();
107
108     System.out.println(x: "Patient Report from Lab B:");
109     System.out.println(patientReportB.getPatientReport());
110     System.out.println();
111
112     System.out.println(x: "Patient Report from Lab C:");
113     System.out.println(patientReportC.getPatientReport());
114     System.out.println();
115
116     System.out.println(x: "===== ADVANTAGES =====");
117     System.out.println(x: "1. Unified Interface: Hospital system can use single getPatientReport() method");
118     System.out.println(x: "2. No Modification: Existing lab classes remain unchanged");
119     System.out.println(x: "3. Flexible: Easy to add new labs by creating new adapters");
120     System.out.println(x: "4. Decoupling: Hospital system decoupled from lab implementations");
121 }
122

```

op:



The screenshot shows an IDE window with a project named PESIU23CS658. The left sidebar displays a file explorer with a folder named 'gaming_application' containing files like RealVideo.class, Video.class, VideoCache.class, VideoProxy.class, VideoStreamingProxy.class, and VideoStreamingProxy.java. The main editor shows the RealVideo.class file with a single line of code: `System.out.println("Video loaded successfully: " + this.name);`. The bottom panel is divided into 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'TERMINAL'. The 'TERMINAL' tab is active, showing the output of a Java command. The output includes a usage message, an error message 'Error: Could not find or load main class HospitalManagementSystemDemo', and a stack trace indicating a 'java.lang.ClassNotFoundException: HospitalManagementSystemDemo'. Below the error, the terminal shows the execution of a Java command to run the 'hospital_management_system' application, which outputs a series of patient reports from Lab A, Lab B, and Lab C, followed by a list of advantages of the adapter pattern.

```
Usage: javac <options> <source files>
use --help for a list of possible options
Error: Could not find or load main class HospitalManagementSystemDemo
Caused by: java.lang.ClassNotFoundException: HospitalManagementSystemDemo
PS C:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\PESIU23CS658\hospital_management_system> cd "c:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\
PESIU23CS658\hospital_management_system" ; javac LabAdapter.java ; java LabAdapter
===== HOSPITAL MANAGEMENT SYSTEM =====
Pattern: ADAPTER PATTERN
Location: hospital_management_system
SRN: PESIU23CS658

Patient Report from Lab A:
Lab A - Blood Report: RBC: 4.5M/µL, WBC: 7K/µL, Hemoglobin: 13.5g/dL

Patient Report from Lab B:
Lab B - Results: Glucose: 95mg/dL, Cholesterol: 180mg/dL, Triglycerides: 180mg/dL

Patient Report from Lab C:
Lab C - Diagnostics: BP: 120/80, Heart Rate: 72bpm, Temperature: 98.6F

===== ADVANTAGES =====
1. Unified Interface: Hospital system can use single getPatientReport() method
2. No Modification: Existing lab classes remain unchanged
3. Flexible: Easy to add new labs by creating new adapters
4. Decoupling: Hospital system decoupled from lab implementations
PS C:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\PESIU23CS658\hospital_management_system>
```

Problem 2

```
J LabCmpl.class J VideoStreamingProxy$VideoCache.class X J VideoStreamingProxy$Video.class J VideoProxy.class J LabAlmpl.class
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 import java.util.HashMap;
3 import java.util.Map;
4
5 public class VideoStreamingProxy {
6     public VideoStreamingProxy() {
7     }
8
9     public static void main(String[] var0) {
10         System.out.println("===== ONLINE VIDEO STREAMING PLATFORM =====");
11         System.out.println("Pattern: PROXY PATTERN");
12         System.out.println("Location: online_video_streaming_platform");
13         System.out.println("SRN: PES1U23CS658");
14         System.out.println();
15         System.out.println("--- Scenario 1: User 1 requests Movie A ---");
16         Video var1 = VideoStreamingProxy.VideoCache.getVideo("The Matrix");
17         System.out.println(var1.getInfo());
18         System.out.println("User 1 starts playback:");
19         var1.play();
20         System.out.println();
21         System.out.println("--- Scenario 2: User 2 requests same Movie (from cache) ---");
22         long var2 = System.currentTimeMillis();
23         Video var4 = VideoStreamingProxy.VideoCache.getVideo("The Matrix");
24         long var5 = System.currentTimeMillis();
25         System.out.println("Time to retrieve: " + (var5 - var2) + "ms (uses cache, no reload!)");
26         System.out.println(var4.getInfo());
27         System.out.println("User 2 starts playback:");
28         var4.play();
29         System.out.println();
30         System.out.println("--- Scenario 3: User 3 requests different Movie ---");
31         Video var7 = VideoStreamingProxy.VideoCache.getVideo("Inception");
32         System.out.println(var7.getInfo());
33         System.out.println("User 3 starts playback:");
34         var7.play();
35         System.out.println();
36         System.out.println("--- Scenario 4: Multiple quick accesses to cached videos ---");
37
38         for(int var8 = 1; var8 <= 3; ++var8) {
39             System.out.println("User " + var8 + " playing The Matrix:");
40             Video var9 = VideoStreamingProxy.VideoCache.getVideo("The Matrix");
41             var9.play();
42         }
43
44         System.out.println();
45         System.out.println("===== ADVANTAGES =====");
46         System.out.println("1. Lazy Loading: Videos loaded only when playback starts");
47     }
}
```

```
J LabCimpl.class  J VideoStreamingProxy$VideoCache.class X  J VideoStreamingProxy$Video.class  J VideoProxy.class  J LabAlmpl.class  ...
45 System.out.println("===== ADVANTAGES =====");
46 System.out.println("1. Lazy Loading: Videos loaded only when playback starts");
47 System.out.println("2. Performance: Same video reused without reloading");
48 System.out.println("3. Reduced Bandwidth: No repeated downloads of same content");
49 System.out.println("4. Resource Efficient: Memory and CPU optimized");
50 System.out.println("5. Access Control: Proxy can implement authentication/logging");
51 System.out.println("Total cached videos: " + VideoStreamingProxy.VideoCache.getCacheSize());
52 }
53
54 static class RealVideo implements Video {
55     private String name;
56
57     public RealVideo(String var1) {
58         this.name = var1;
59         this.loadVideo();
60     }
61
62     private void loadVideo() {
63         System.out.println("Loading high-resolution video: " + this.name + " (This takes time...)");
64
65         try {
66             Thread.sleep(2000L);
67         } catch (InterruptedException var2) {
68             Thread.currentThread().interrupt();
69         }
70
71         System.out.println("Video loaded successfully: " + this.name);
72     }
73
74     public void play() {
75         System.out.println("Playing video: " + this.name);
76     }
77
78     public String getInfo() {
79         return "Video: " + this.name + " (4K, 25GB)";
80     }
81 }
82
83 static class InternalVideoProxy implements Video {
84     private String name;
85     private RealVideo realVideo;
86
87     public InternalVideoProxy(String var1) {
88         this.name = var1;
89         this.realVideo = null;
90     }
91 }
```

Ln 1, Col 1

```
J LabCmpl.class  J VideoStreamingProxy$VideoCache.class X  J VideoStreamingProxy$Video.class  J VideoProxy.class  J LabAlmpl.c
86
87     public InternalVideoProxy(String var1) {
88         this.name = var1;
89         this.realVideo = null;
90     }
91
92     public void play() {
93         if (this.realVideo == null) {
94             this.realVideo = new RealVideo(this.name);
95         }
96
97         this.realVideo.play();
98     }
99
100     public String getInfo() {
101         return "Video: " + this.name + " (4K, 25GB)";
102     }
103 }
104
105 static class VideoCache {
106     private static Map<String, Video> cache = new HashMap();
107
108     VideoCache() {
109     }
110
111     public static Video getVideo(String var0) {
112         if (!cache.containsKey(var0)) {
113             cache.put(var0, new InternalVideoProxy(var0));
114             System.out.println("Created new proxy for: " + var0);
115         } else {
116             System.out.println("Retrieved from cache: " + var0);
117         }
118
119         return (Video)cache.get(var0);
120     }
121
122     public static int getCacheSize() {
123         return cache.size();
124     }
125 }
126
127 interface Video {
128     void play();
129
130     String getInfo();
131 }
132 }
```



```

J RealVideo.class  J LabAdapter.java  J VideoStreamingProxy.java  X
online_video_streaming_platform > J VideoStreamingProxy.java > Language Support for Java(TM) by Red Hat > Video
1 // Proxy Pattern - Online Video Streaming Platform
2 // Design Pattern for lazy loading and caching of high-resolution videos
3
4 // Step 1: Create Video interface
5 interface Video {
6     void play();
7     String getInfo();
8 }
9
10 // Step 2: Create Real Subject - Actual video that requires expensive loading
11 class RealVideo implements Video {
12     private String name;
13
14     public RealVideo(String name) {
15         this.name = name;
16         loadVideo(); // Expensive operation
17     }
18
19     private void loadVideo() {
20         System.out.println("Loading high-resolution video: " + name + " (This takes time...)");
21         try {
22             Thread.sleep(millis: 2000); // Simulate loading delay
23         } catch (InterruptedException e) {
24             Thread.currentThread().interrupt();
25         }
26         System.out.println("Video loaded successfully: " + name);
27     }
28
29     @Override
30     public void play() {
31         System.out.println("Playing video: " + name);
32     }
33
34     @Override
35     public String getInfo() {
36         return "Video: " + name + " (4K, 25GB)";
37     }
38 }
39
40 // Step 3: Create Proxy - Controls access and implements caching
41 class VideoProxy implements Video {
42     private String name;
43     private RealVideo realVideo;
44
45     public VideoProxy(String name) {
46         this.name = name;
47         this.realVideo = null; // Lazy initialization
48     }
49
50     @Override
51     public void play() {
52         // Load video only when needed (lazy loading)
53         if (realVideo == null) {
54             realVideo = new RealVideo(name);
55         }
56     }
57 }

```

```
online_video_streaming_platform > J VideoStreamingProxy.java > Language Support for Java(TM) by Red Hat > Video
86 public class VideoStreamingProxy {
87     public static void main(String[] args) {
88         System.out.println(x: "Pattern: PROXY PATTERN");
89         System.out.println(x: "Location: online_video_streaming_platform");
90         System.out.println(x: "SRN: PESIU23CS658");
91         System.out.println();
92
93         // Scenario 1: First request for a video (loads video)
94         System.out.println(x: "---- Scenario 1: User 1 requests Movie A ----");
95         Video video1 = VideoCache.getVideo(name: "The Matrix");
96         System.out.println(video1.getInfo());
97         System.out.println(x: "User 1 starts playback:");
98         video1.play();
99         System.out.println();
100
101         // Scenario 2: Second request for same video (uses cache, no reload)
102         System.out.println(x: "---- Scenario 2: User 2 requests same Movie (from cache) ----");
103         long startTime = System.currentTimeMillis();
104         Video video2 = VideoCache.getVideo(name: "The Matrix");
105         long endTime = System.currentTimeMillis();
106         System.out.println("Time to retrieve: " + (endTime - startTime) + "ms (uses cache, no reload!)");
107         System.out.println(video2.getInfo());
108         System.out.println(x: "User 2 starts playback:");
109         video2.play();
110         System.out.println();
111
112         // Scenario 3: Third request for different video
113         System.out.println(x: "---- Scenario 3: User 3 requests different Movie ----");
114         Video video3 = VideoCache.getVideo(name: "Inception");
115         System.out.println(video3.getInfo());
116         System.out.println(x: "User 3 starts playback:");
117         video3.play();
118         System.out.println();
119
120         // Scenario 4: Multiple accesses to same cached video
121         System.out.println(x: "---- Scenario 4: Multiple quick accesses to cached videos ----");
122         for (int i = 1; i <= 3; i++) {
123             System.out.println("User " + i + " playing The Matrix:");
124             Video v = VideoCache.getVideo(name: "The Matrix");
125             v.play();
126         }
127         System.out.println();
128
129         System.out.println(x: "***** ADVANTAGES *****");
130         System.out.println(x: "1. Lazy Loading: Videos loaded only when playback starts");
131         System.out.println(x: "2. Performance: Same video reused without reloading");
132         System.out.println(x: "3. Reduced Bandwidth: No repeated downloads of same content");
133         System.out.println(x: "4. Resource Efficient: Memory and CPU optimized");
134         System.out.println(x: "5. Access Control: Proxy can implement authentication/logging");
135         System.out.println("Total cached videos: " + VideoCache.getCachedSize());
136     }
137 }
138
139
```

op:

```
File Edit Selection View Go Run ... PESIU23CS658 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
> e_commerce_platform
> gaming_application
> hospital_management_system
> online_video_streaming_platform
  > gaming_application
    J RealVideo.class
    F SOLUTION_DOCUMENTATION.txt
    J Video.class
    J VideoCache.class
    J VideoProxy.class
    J VideoStreamingProxy.class
    J VideoStreamingProxy.java
    F COMPLETE_SOLUTIONS_SUMMARY.txt

PS C:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\PESIU23CS658\hospital_management_system> cd "C:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\
PESIU23CS658\online_video_streaming_platform" ; javac VideoStreamingProxy.java ; java VideoStreamingProxy

---- Scenario 1: User 1 requests Movie A ----
Created new proxy for: The Matrix
Video: The Matrix (4K, 25GB)
User 1 starts playback:
Loading high-resolution video: The Matrix (This takes time...)
Video loaded successfully: The Matrix
Playing video: The Matrix

---- Scenario 2: User 2 requests same Movie (from cache) ----
Retrieved from cache: The Matrix
Time to retrieve: 0ms (uses cache, no reload!)
Video: The Matrix (4K, 25GB)
User 2 starts playback:
Playing video: The Matrix

---- Scenario 3: User 3 requests different Movie ----
Created new proxy for: Inception
Video: Inception (4K, 25GB)
User 3 starts playback:
Loading high-resolution video: Inception (This takes time...)
Video loaded successfully: Inception
Playing video: Inception

---- Scenario 4: Multiple quick accesses to cached videos ----
User 1 playing The Matrix:
Retrieved from cache: The Matrix
Playing video: The Matrix
User 2 playing The Matrix:
Retrieved from cache: The Matrix
Playing video: The Matrix
User 3 playing The Matrix:
Retrieved from cache: The Matrix
Playing video: The Matrix

***** ADVANTAGES *****
1. Lazy Loading: Videos loaded only when playback starts
2. Performance: Same video reused without reloading
3. Reduced Bandwidth: No repeated downloads of same content
4. Resource Efficient: Memory and CPU optimized
5. Access Control: Proxy can implement authentication/logging
Total cached videos: 2
PS C:\Users\Trishul Swamy HM\Downloads\SEM6\OOAD\PESIU23CS658\online_video_streaming_platform>
```

Problem 3

```
J LabCmpl.class  J VideoStreamingProxy$VideoCache.class  J TreeProperties.class  J GamingFlyweight$TreeProperties.class X  J VideoStre
1 // Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).
2 import java.io.PrintStream;
3 import java.util.ArrayList;
4 import java.util.HashMap;
5 import java.util.List;
6 import java.util.Map;
7
8 public class GamingFlyweight {
9     public GamingFlyweight() {
10    }
11
12    public static void main(String[] var0) {
13        System.out.println("===== GAMING APPLICATION =====");
14        System.out.println("Pattern: FLYWEIGHT PATTERN");
15        System.out.println("Location: gaming_application");
16        System.out.println("SRN: PES1U23CS658");
17        System.out.println();
18        GameEnvironment var1 = new GameEnvironment();
19        System.out.println("--- Creating Game Objects (1000 objects) ---");
20        System.out.println("\nAdding Oak Trees (300 instances):");
21        TreeProperties var2 = GamingFlyweight.TreeFactory.getTreeProperties("Oak", "Green", "barkTexture", 40);
22
23        for(int var3 = 0; var3 < 300; ++var3) {
24            var1.addTree(var2, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F));
25        }
26
27        System.out.println("Added 300 Oak trees using 1 shared property object");
28        System.out.println("\nAdding Pine Trees (250 instances):");
29        TreeProperties var12 = GamingFlyweight.TreeFactory.getTreeProperties("Pine", "DarkGreen", "needleTexture", 50);
30
31        for(int var4 = 0; var4 < 250; ++var4) {
32            var1.addTree(var12, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F));
33        }
34
35        System.out.println("Added 250 Pine trees using 1 shared property object");
36        System.out.println("\nAdding Gray Rocks (200 instances):");
37        TreeProperties var13 = GamingFlyweight.RockFactory.getRockProperties("Large", "Gray");
38
39        for(int var5 = 0; var5 < 200; ++var5) {
40            var1.addRock(var13, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F));
41        }
42
43        System.out.println("Added 200 Gray rocks using 1 shared property object");
44        System.out.println("\nAdding Brown Rocks (150 instances):");
45        TreeProperties var14 = GamingFlyweight.RockFactory.getRockProperties("Small", "Brown");
46    }
```

```
J LabCmpl.class J VideoStreamingProxy$VideoCache.class X J TreeProperties.class J GamingFlyweight$TreeProperties.class X J VideoStre
45 TreeProperties var14 = GamingFlyweight.RockFactory.getRockProperties("Small", "Brown");
46
47 for(int var6 = 0; var6 < 150; ++var6) {
48     var1.addRock(var14, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F),
49 }
50
51 System.out.println("Added 150 Brown rocks using 1 shared property object");
52 System.out.println("\nAdding Residential Buildings (150 instances):");
53 TreeProperties var15 = GamingFlyweight.BuildingFactory.getBuildingProperties("Residential", "Concrete");
54
55 for(int var7 = 0; var7 < 150; ++var7) {
56     var1.addBuilding(var15, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F),
57 }
58
59 System.out.println("Added 150 Residential buildings using 1 shared property object");
60 System.out.println("\nAdding Commercial Buildings (100 instances):");
61 TreeProperties var16 = GamingFlyweight.BuildingFactory.getBuildingProperties("Commercial", "Steel");
62
63 for(int var8 = 0; var8 < 100; ++var8) {
64     var1.addBuilding(var16, (int)(Math.random() * (double)1000.0F), (int)(Math.random() * (double)1000.0F),
65 }
66
67 System.out.println("Added 100 Commercial buildings using 1 shared property object");
68 System.out.println("\n===== MEMORY OPTIMIZATION STATISTICS =====");
69 System.out.println("Total Game Objects Created: " + var1.getTotalObjectCount());
70 System.out.println("Total Unique Tree Properties: " + GamingFlyweight.TreeFactory.getTotalFlyweightsCreated());
71 System.out.println("Total Unique Rock Properties: " + GamingFlyweight.RockFactory.getTotalRocksCreated());
72 System.out.println("Total Unique Building Properties: " + GamingFlyweight.BuildingFactory.getTotalBuildingsCreated());
73 int var17 = GamingFlyweight.TreeFactory.getTotalFlyweightsCreated() + GamingFlyweight.RockFactory.getTotalRocksCreated() + GamingFlyweight.BuildingFactory.getTotalBuildingsCreated();
74 System.out.println("\nTotal Shared Property Objects: " + var17);
75 System.out.println("Memory Saved: " + (var1.getTotalObjectCount() - var17) + " property objects not duplicated");
76 PrintStream var10000 = System.out;
77 Object[] var10002 = new Object[] {(double)(var1.getTotalObjectCount() - var17) / (double)var1.getTotalObjectCount()};
78 var10000.println("Efficiency: " + String.format("%.2f%%", var10002) + " memory reduction");
79 System.out.println("\n===== SAMPLE RENDERING =====");
80 System.out.println("Showing first 3 objects of each type:");
81 System.out.println("\n--- Trees (first 3) ---");
82 new ArrayList();
83 int var10 = 0;
84
85 for(int var11 = 0; var11 < 900; ++var11) {
86     if (var10 < 3) {
87         System.out.println("Tree " + (var11 + 1) + ": Oak or Pine at random position");
88         ++var10;
89     }
90 }
91
```

```

89     }
90 }
91
92 System.out.println("\n===== ADVANTAGES =====");
93 System.out.println("1. Memory Efficiency: 1000 objects stored with only " + var17 + " property sets");
94 System.out.println("2. Reduced GPU Load: Fewer texture uploads and material buffers");
95 System.out.println("3. Faster Rendering: Shared properties cached in GPU memory");
96 System.out.println("4. Scalability: Can handle thousands of objects without memory issues");
97 System.out.println("5. Performance: Reduced cache misses due to data locality");
98 }
99
100 static class TreeProperties {
101     private String texture;
102     private String color;
103     private String type;
104     private int height;
105
106     public TreeProperties(String var1, String var2, String var3, int var4) {
107         this.texture = var1;
108         this.color = var2;
109         this.type = var3;
110         this.height = var4;
111     }
112
113     public String getInfo() {
114         return "Type: " + this.type + ", Color: " + this.color + ", Texture: " + this.texture + ", Height: "
115     }
116
117     public String toString() {
118         return this.type + "(" + this.color + ")";
119     }
120 }
121
122 static class TreeInstance {
123     private TreeProperties properties;
124     private int xPosition;
125     private int yPosition;
126     private int zPosition;
127
128     public TreeInstance(TreeProperties var1, int var2, int var3, int var4) {
129         this.properties = var1;
130         this.xPosition = var2;
131         this.yPosition = var3;
132         this.zPosition = var4;
133     }
134
135     public String render() {

```

```

137     return "Rendering " + var10000 + " at position (" + this.xPosition + ", " + this.yPosition + ", " +
138 }
139
140 public String getDetails() {
141     String var10000 = this.properties.getInfo();
142     return var10000 + " at (" + this.xPosition + ", " + this.yPosition + ", " + this.zPosition + ")";
143 }
144 }
145
146 static class TreeFactory {
147     private static Map<String, TreeProperties> flyweights = new HashMap();
148
149     TreeFactory() {
150     }
151
152     public static TreeProperties getTreeProperties(String var0, String var1, String var2, int var3) {
153         String var4 = var0 + "_" + var1 + "_" + var2 + "_" + var3;
154         if (!flyweights.containsKey(var4)) {
155             flyweights.put(var4, new TreeProperties(var2, var1, var0, var3));
156             System.out.println("[Factory] Created new shared properties: " + var4);
157         } else {
158             System.out.println("[Factory] Reusing existing properties: " + var4);
159         }
160
161         return (TreeProperties)flyweights.get(var4);
162     }
163
164     public static int getTotalFlyweightsCreated() {
165         return flyweights.size();
166     }
167 }
168
169 static class RockFactory {
170     private static Map<String, TreeProperties> flyweights = new HashMap();
171
172     RockFactory() {
173     }
174
175     public static TreeProperties getRockProperties(String var0, String var1) {
176         String var2 = "Rock_" + var0 + "_" + var1;
177         if (!flyweights.containsKey(var2)) {
178             flyweights.put(var2, new TreeProperties("rockTexture", var1, var0, 1));
179             System.out.println("[RockFactory] Created new rock: " + var2);
180         } else {
181             System.out.println("[RockFactory] Reusing rock: " + var2);
182         }

```



```
J LabCimpl.class  J VideoStreamingProxy$VideoCache.class  J TreeProperties.class  J GamingFlyweight$TreeProperties.class  J VideoStreamingProxy$Video.class

197
198
199 public static TreeProperties getBuildingProperties(String var0, String var1) {
200     String var2 = "Building_" + var0 + "_" + var1;
201     if (!flyweights.containsKey(var2)) {
202         flyweights.put(var2, new TreeProperties("concrete", "gray", var0, 50));
203         System.out.println("[BuildingFactory] Created new building: " + var2);
204     } else {
205         System.out.println("[BuildingFactory] Reusing building: " + var2);
206     }
207     return (TreeProperties)flyweights.get(var2);
208 }
209
210 public static int getTotalBuildingsCreated() {
211     return flyweights.size();
212 }
213 }
214
215 static class GameEnvironment {
216     private List<TreeInstance> trees = new ArrayList();
217     private List<TreeInstance> rocks = new ArrayList();
218     private List<TreeInstance> buildings = new ArrayList();
219
220     GameEnvironment() {
221     }
222
223     public void addTree(TreeProperties var1, int var2, int var3, int var4) {
224         this.trees.add(new TreeInstance(var1, var2, var3, var4));
225     }
226
227     public void addRock(TreeProperties var1, int var2, int var3, int var4) {
228         this.rocks.add(new TreeInstance(var1, var2, var3, var4));
229     }
230
231     public void addBuilding(TreeProperties var1, int var2, int var3, int var4) {
232         this.buildings.add(new TreeInstance(var1, var2, var3, var4));
233     }
234
235     public void render() {
236         System.out.println("\n--- Rendering Trees ---");
237
238         for(TreeInstance var2 : this.trees) {
239             System.out.println(var2.render());
240         }
241
242         System.out.println("\n--- Rendering Rocks ---");
243
244         for(TreeInstance var5 : this.rocks) {
245             System.out.println(var5.render());
246         }
247
248         System.out.println("\n--- Rendering Buildings ---");
249
250         for(TreeInstance var6 : this.buildings) {
251             System.out.println(var6.render());
252         }
253     }
254
255     public int getTotalObjectCount() {
256         return this.trees.size() + this.rocks.size() + this.buildings.size();
257     }
258 }
259
260 }
261
```

OUTPUT:

The screenshot shows an IDE with a project named 'PES1U23C5658'. The project structure on the left includes:

- e-commerce_platform
- gaming_application
- hospital_management_system
- online_video_streaming_platform
- gaming_application
 - RealVideo.class
 - SOLUTION_DOCUMENTATION.txt
 - Video.class
 - VideoCache.class
 - VideoProxy.class
 - VideoStreamingProxy.class
 - VideoStreamingProxy.java
- COMPLETE_SOLUTIONS_SUMMARY.txt

The terminal output shows the execution of a Java application. The command executed is:

```
PS C:\Users\Trishul Swamy HP\Downloads\SEM6\OOD\PES1U23C5658\online_video_streaming_platform> cd "C:\Users\Trishul Swamy HP\Down  
loads\SEM6\OOD\PES1U23C5658\gaming_application"; javac GamingFlyweight.java ; java GamingFlyweight
```

The output shows the creation of 300 Oak trees using 1 shared property object. It also shows the creation of 250 Pine trees using 1 shared property object, 200 Gray rocks using 1 shared property object, 150 Brown rocks using 1 shared property object, 150 Residential buildings using 1 shared property object, and 100 Commercial buildings using 1 shared property object.

----- MEMORY OPTIMIZATION STATISTICS -----

```
Total Game Objects Created: 1150  
Total Unique Tree Properties: 2  
Total Unique Rock Properties: 2  
Total Unique Building Properties: 2  
  
Total Shared Property Objects: 6  
Memory Saved: 1144 property objects not duplicated  
Efficiency: 99.48% memory reduction
```

----- SAMPLE RENDERING -----

```
Showing first 3 objects of each type:  
  
--- Trees (first 3) ---  
Tree 1: Oak or Pine at random position  
Tree 2: Oak or Pine at random position  
Tree 3: Oak or Pine at random position
```

----- ADVANTAGES -----

```
1. Memory Efficiency: 1000 objects stored with only 6 property sets  
2. Reduced GPU Load: Fewer texture uploads and material buffers  
3. Faster Rendering: Shared properties cached in GPU memory  
4. Scalability: Can handle thousands of objects without memory issues  
5. Performance: Reduced cache misses due to data locality
```

3 OP ABOVE:

Problem 4:


```
J ShippingService.class J ECommerceFacadeDemo.java 9+ X J ProductCatalog$Product.class J ECommerceFacadeDemo.class J InventoryService.class J OrderService.class J
e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo
1 // Facade Pattern - E-commerce Platform
2 // Design Pattern to simplify complex subsystems into a single interface
3
4 // Step 1: Create complex subsystems that need coordination
5
6 public class ECommerceFacadeDemo {
7     // Authentication Subsystem
8     static class AuthenticationService {
9         public boolean login(String username, String password) {
10             System.out.println("[AuthService] Verifying credentials for: " + username);
11             System.out.println("[AuthService] User '" + username + "' logged in successfully");
12             return true;
13         }
14
15         public void logout(String username) {
16             System.out.println("[AuthService] User '" + username + "' logged out");
17         }
18     }
19
20     // Product Catalog Subsystem
21     static class ProductCatalog {
22         static class Product {
23             String id;
24             String name;
25             double price;
26             int stock;
27
28             Product(String id, String name, double price, int stock) {
29                 this.id = id;
30                 this.name = name;
31                 this.price = price;
32                 this.stock = stock;
33             }
34         }
35
36         private java.util.Map<String, Product> products = new java.util.HashMap<>();
37
38         public ProductCatalog() {
39             products.put(key: "P001", new Product(id: "P001", name: "Laptop", price: 999.99, stock: 10));
40             products.put(key: "P002", new Product(id: "P002", name: "Mouse", price: 29.99, stock: 50));
41             products.put(key: "P003", new Product(id: "P003", name: "Keyboard", price: 79.99, stock: 30));
42         }
43
44         public Product selectProduct(String productId) {
45             Product product = products.get(productId);
46             if (product != null && product.stock > 0) {
47                 System.out.println("[Catalog] Selected product: " + product.name + " - $" + product.price);
48                 return product;
49             } else {
50                 System.out.println("[Catalog] Product out of stock: " + productId);
51                 return null;
52             }
53         }
54     }
55 }
```

```

e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo
6 public class ECommerceFacadeDemo {
57     static class PaymentService {
58         public boolean processPayment(String cardNumber, String cardholderName, double amount) {
61             if (amount < 50000) {
62                 System.out.println(x: "[PaymentService] Payment authorized successfully");
63                 return true;
64             } else {
65                 System.out.println(x: "[PaymentService] Payment declined (amount exceeds limit)");
66                 return false;
67             }
68         }
69     }
70 }
71
72 // Inventory Subsystem
73 static class InventoryService {
74     public boolean updateInventory(String productId, int quantity) {
75         System.out.println("[Inventory] Updating stock for product: " + productId);
76         System.out.println("[Inventory] Reducing quantity by: " + quantity);
77         System.out.println(x: "[Inventory] Stock updated successfully");
78         return true;
79     }
80 }
81
82 // Order Management Subsystem
83 static class OrderService {
84     private int orderCounter = 1000;
85
86     public String createOrder(String productId, int quantity, double totalPrice) {
87         orderCounter++;
88         String orderId = "ORD_" + orderCounter;
89         System.out.println("[OrderService] Creating order: " + orderId);
90         System.out.println("[OrderService] Product: " + productId + ", Quantity: " + quantity + ", Total: $" + totalPrice);
91         return orderId;
92     }
93 }
94
95 // Shipping Subsystem
96 static class ShippingService {
97     public void trackOrder(String orderId) {
98         System.out.println("[Shipping] Tracking order: " + orderId);
99         System.out.println(x: "[Shipping] Status: Order confirmed and shipped");
100        System.out.println(x: "[Shipping] Estimated Delivery: 3-5 business days");
101    }
102
103    public void generateShippingLabel(String orderId) {
104        System.out.println("[Shipping] Generated shipping label for: " + orderId);
105    }
106 }
107
108 // Step 2: Create Facade - Simplifies complex workflow
109 static class ECommerceFacade {
110     private AuthenticationService authService;
111     private ProductCatalog catalog;
112     private PaymentService paymentService;

```

e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo

```
6 public class ECommerceFacadeDemo {
109 static class ECommerceFacade {
113     private InventoryService inventoryService;
114     private OrderService orderService;
115     private ShippingService shippingService;
116
117     private String currentUser;
118     private ProductCatalog.Product selectedProduct;
119     private String orderId;
120
121     public ECommerceFacade() {
122         this.authService = new AuthenticationService();
123         this.catalog = new ProductCatalog();
124         this.paymentService = new PaymentService();
125         this.inventoryService = new InventoryService();
126         this.orderService = new OrderService();
127         this.shippingService = new ShippingService();
128     }
129
130     public void completePurchase(String username, String password, String productId,
131                                int quantity, String cardNumber, String cardholderName) {
132         System.out.println(x: "\n===== INITIATING PURCHASE PROCESS =====\n");
133
134         if (!login(username, password)) {
135             System.out.println(x: "ERROR: Login failed!");
136             return;
137         }
138
139         if (!selectProduct(productId, quantity)) {
140             System.out.println(x: "ERROR: Product selection failed!");
141             return;
142         }
143
144         double totalAmount = selectedProduct.price * quantity;
145         if (!processPayment(cardNumber, cardholderName, totalAmount)) {
146             System.out.println(x: "ERROR: Payment processing failed!");
147             return;
148         }
149
150         if (!updateInventory(productId, quantity)) {
151             System.out.println(x: "ERROR: Inventory update failed!");
152             return;
153         }
154
155         orderId = createOrder(productId, quantity, totalAmount);
156         trackOrder(orderId);
157
158         System.out.println(x: "\n===== PURCHASE COMPLETED SUCCESSFULLY =====\n");
159     }
160
161     private boolean login(String username, String password) {
162         System.out.println(x: "\n--- Step 1: LOGIN ---");
163         currentUser = username;
164         return authService.login(username, password);
165     }
}
```

```
e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo
6  v public class ECommerceFacadeDemo {
109 v static class ECommerceFacade {
166
167     private boolean selectProduct(String productId, int qty) {
168         System.out.println(x: "\n--- Step 2: SELECT PRODUCT ---");
169         selectedProduct = catalog.selectProduct(productId);
170         return selectedProduct != null;
171     }
172
173     private boolean processPayment(String cardNumber, String cardholderName, double amount) {
174         System.out.println(x: "\n--- Step 3: MAKE PAYMENT ---");
175         return paymentService.processPayment(cardNumber, cardholderName, amount);
176     }
177
178     private boolean updateInventory(String productId, int quantity) {
179         System.out.println(x: "\n--- Step 4: UPDATE INVENTORY ---");
180         return inventoryService.updateInventory(productId, quantity);
181     }
182
183     private String createOrder(String productId, int quantity, double totalAmount) {
184         System.out.println(x: "\n--- Step 5: CREATE ORDER ---");
185         return orderService.createOrder(productId, quantity, totalAmount);
186     }
187
188     private void trackOrder(String orderId) {
189         System.out.println(x: "\n--- Step 6: TRACK ORDER ---");
190         shippingService.generateShippingLabel(orderId);
191         shippingService.trackOrder(orderId);
192     }
193
194     public void logout() {
195         System.out.println(x: "\n--- LOGOUT ---");
196         authService.logout(currentUser);
197     }
198 }
199
200 // Step 3: Main class to demonstrate the pattern
201 Run main | Debug main | Run | Debug
202 public static void main(String[] args) {
203     System.out.println(x: "===== E-COMMERCE PLATFORM =====");
204     System.out.println(x: "Pattern: FACADE PATTERN");
205     System.out.println(x: "Location: e_commerce_platform");
206     System.out.println(x: "SRN: PES1U23CS658");
207     System.out.println();
208
209     // Create facade
210     ECommerceFacade ecommerce = new ECommerceFacade();
211
212     System.out.println(x: "===== WITHOUT FACADE (Complex approach) =====");
213     System.out.println(x: "User would need to manually call:");
214     System.out.println(x: "1. authService.login(username, password)");
215     System.out.println(x: "2. catalog.selectProduct(productId)");
216     System.out.println(x: "3. paymentService.processPayment(card, amount)");
217     System.out.println(x: "4. inventoryService.updateInventory(productId, qty)");
218     System.out.println(x: "5. orderService.createOrder(productId, qty, amount)");
219 }
```

```
e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo
6  v public class ECommerceFacadeDemo {
109 v static class ECommerceFacade {
166
167     private boolean selectProduct(String productId, int qty) {
168         System.out.println(x: "\n--- Step 2: SELECT PRODUCT ---");
169         selectedProduct = catalog.selectProduct(productId);
170         return selectedProduct != null;
171     }
172
173     private boolean processPayment(String cardNumber, String cardholderName, double amount) {
174         System.out.println(x: "\n--- Step 3: MAKE PAYMENT ---");
175         return paymentService.processPayment(cardNumber, cardholderName, amount);
176     }
177
178     private boolean updateInventory(String productId, int quantity) {
179         System.out.println(x: "\n--- Step 4: UPDATE INVENTORY ---");
180         return inventoryService.updateInventory(productId, quantity);
181     }
182
183     private String createOrder(String productId, int quantity, double totalAmount) {
184         System.out.println(x: "\n--- Step 5: CREATE ORDER ---");
185         return orderService.createOrder(productId, quantity, totalAmount);
186     }
187
188     private void trackOrder(String orderId) {
189         System.out.println(x: "\n--- Step 6: TRACK ORDER ---");
190         shippingService.generateShippingLabel(orderId);
191         shippingService.trackOrder(orderId);
192     }
193
194     public void logout() {
195         System.out.println(x: "\n--- LOGOUT ---");
196         authService.logout(currentUser);
197     }
198 }
199
200 // Step 3: Main class to demonstrate the pattern
201 Run main | Debug main | Run | Debug
202 public static void main(String[] args) {
203     System.out.println(x: "===== E-COMMERCE PLATFORM =====");
204     System.out.println(x: "Pattern: FACADE PATTERN");
205     System.out.println(x: "Location: e_commerce_platform");
206     System.out.println(x: "SRN: PES1U23CS658");
207     System.out.println();
208
209     // Create facade
210     ECommerceFacade ecommerce = new ECommerceFacade();
211
212     System.out.println(x: "===== WITHOUT FACADE (Complex approach) =====");
213     System.out.println(x: "User would need to manually call:");
214     System.out.println(x: "1. authService.login(username, password)");
215     System.out.println(x: "2. catalog.selectProduct(productId)");
216     System.out.println(x: "3. paymentService.processPayment(card, amount)");
217     System.out.println(x: "4. inventoryService.updateInventory(productId, qty)");
218     System.out.println(x: "5. orderService.createOrder(productId, qty, amount)");
219 }
```

```
o Run ... PES1U23CS658
J ShippingService.class J ECommerceFacadeDemo.java 9+ X J ProductCatalog$Product.class J ECommerceFacadeDemo.class J InventoryService.class J OrderService.class
e_commerce_platform > J ECommerceFacadeDemo.java > Language Support for Java(TM) by Red Hat > ECommerceFacadeDemo
1 // Facade Pattern - E-commerce Platform
2 // Design Pattern to simplify complex subsystems into a single interface
3
4 // Step 1: Create complex subsystems that need coordination
5
6 public class ECommerceFacadeDemo {
7     // Authentication Subsystem
8     static class AuthenticationService {
9         public boolean login(String username, String password) {
10             System.out.println("[AuthService] Verifying credentials for: " + username);
11             System.out.println("[AuthService] User " + username + " logged in successfully");
12             return true;
13         }
14
15         public void logout(String username) {
16             System.out.println("[AuthService] User " + username + " logged out");
17         }
18     }
19
20     // Product Catalog Subsystem
21     static class ProductCatalog {
22         static class Product {
23             String id;
24             String name;
25             double price;
26             int stock;
27
28             Product(String id, String name, double price, int stock) {
29                 this.id = id;
30                 this.name = name;
31                 this.price = price;
32                 this.stock = stock;
33             }
34         }
35
36         private java.util.Map<String, Product> products = new java.util.HashMap<>();
37
38         public ProductCatalog() {
39             products.put(key: "P001", new Product(id: "P001", name: "Laptop", price: 999.99, stock: 10));
40             products.put(key: "P002", new Product(id: "P002", name: "Mouse", price: 29.99, stock: 50));
41             products.put(key: "P003", new Product(id: "P003", name: "Keyboard", price: 79.99, stock: 30));
42         }
43
44         public Product selectProduct(String productId) {
45             Product product = products.get(productId);
46             if (product != null && product.stock > 0) {
47                 System.out.println("[Catalog] Selected product: " + product.name + " - $" + product.price);
48                 return product;
49             } else {
50                 System.out.println("[Catalog] Product out of stock: " + productId);
51                 return null;
52             }
53         }
54     }
55 }
```

OUTPUT:

```
PS C:\Users\Trishul Swamy HM\Downloads\SEM6\00AD\PES1U23CS658\gaming_application> cd "c:\Users\Trishul Swamy HM\Downloads\SEM6\00AD\PES1U23CS658\e_commerce_platform" ; javac eCommerceFacadeDemo.java ; java eCommerceFacadeDemo
Note: Making a bundle purchase

===== INITIATING PURCHASE PROCESS =====

--- Step 1: LOGIN ---
[AuthService] Verifying credentials for: jane smith
[AuthService] User 'jane_smith' logged in successfully

--- Step 2: SELECT PRODUCT ---
[Catalog] Selected product: Mouse - $29.99

--- Step 3: MAKE PAYMENT ---
[PaymentService] Processing payment of $59.98
[PaymentService] Card validation for: Jane Smith
[PaymentService] Payment authorized successfully

--- Step 4: UPDATE INVENTORY ---
[Inventory] Updating stock for product: P002
[Inventory] Reducing quantity by: 2
[Inventory] Stock updated successfully

--- Step 5: CREATE ORDER ---
[OrderService] Creating order: ORD_1002
[OrderService] Product: P002, Quantity: 2, Total: $59.98

--- Step 6: TRACK ORDER ---
[Shipping] Generated shipping label for: ORD_1002
[Shipping] Tracking order: ORD_1002
[Shipping] Status: Order confirmed and shipped
[Shipping] Estimated Delivery: 3-5 business days

===== PURCHASE COMPLETED SUCCESSFULLY =====

--- LOGOUT ---
[AuthService] User 'jane_smith' logged out

===== ADVANTAGES OF FACADE PATTERN =====
1. Simplified Interface: Single completePurchase() method
2. Easy to Use: Clients don't understand complex subsystems
3. Loose Coupling: Subsystems decoupled from client code
4. Maintenance: Easy to modify subsystems without affecting clients
5. Sequential Coordination: Ensures correct order of operations
6. Error Handling: Centralized error handling in facade
7. Reduced Complexity: Hides complex interactions

PS C:\Users\Trishul Swamy HM\Downloads\SEM6\00AD\PES1U23CS658\gaming_application>
```